

```

import { useState, useEffect, useCallback } from "react";
import { createClient } from "https://cdn.jsdelivr.net/npm/@supabase/supabase-js/

// — CONFIG —————
const SUPABASE_URL = "https://vumtiszgrcjzgekkpgmp.supabase.co";
const SUPABASE_ANON_KEY = "sb_publishable_3szZfHs-Yyhg9sGDjprHBg_4SASdoqp";
const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);

// — CONSTANTS —————
const RESPONSABLES = [
  "— COMERCIAL —", "Inside Sales", "Marcell", "Fernanda", "Ana Marieta",
  "— MARKETING —", "Cibele", "Guilherme", "Sarah", "Catharina", "Júlia"
];
const CANAIS = ["Presencial", "VÍdeo", "WhatsApp", "Telefone"];
const PROBABILIDADES = ["Alto", "Médio", "Baixo", "Sem previsão"];
const BLOCOS_COLOR = {
  "Ativação Técnica": "#0ea5e9",
  "Desenvolvimento": "#8b5cf6",
  "Captação": "#10b981",
  "Conversão": "#f59e0b",
  "Financeiro": "#ef4444",
};
const TIER_LABEL = { ST: "Star", 1: "T1", 2: "T2", 3: "T3", 4: "T4", 5: "T5" };
const TIER_COLOR = { ST: "#f59e0b", 1: "#ef4444", 2: "#f97316", 3: "#eab308", 4:

// — STYLES —————
const css = `
  @import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;500;

  * { box-sizing: border-box; margin: 0; padding: 0; }

  body { background: #0a0e1a; }

  :root {
    --bg: #0a0e1a;
    --surface: #111827;
    --surface2: #1a2235;
    --border: #1e2d45;
    --teal: #00bfa5;
    --teal-dim: #00897b;
    --gold: #d4af37;
    --text: #e2e8f0;
    --muted: #64748b;
    --danger: #ef4444;

```

```

    --success: #10b981;
    --warn: #f59e0b;
}

.app {
  font-family: 'DM Sans', sans-serif;
  background: var(--bg);
  color: var(--text);
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

/* HEADER */
.header {
  background: var(--surface);
  border-bottom: 1px solid var(--border);
  padding: 0 1.5rem;
  display: flex;
  align-items: center;
  justify-content: space-between;
  height: 56px;
  position: sticky;
  top: 0;
  z-index: 100;
}

.header-brand {
  display: flex;
  align-items: center;
  gap: 10px;
}

.header-logo {
  width: 28px;
  height: 28px;
  background: linear-gradient(135deg, var(--teal), var(--gold));
  border-radius: 6px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 14px;
  font-weight: 700;
  color: #0a0e1a;
}

.header-title {
  font-size: 15px;
  font-weight: 600;
  color: var(--text);
}

```

```

    letter-spacing: -0.02em;
}
.header-sub {
    font-size: 11px;
    color: var(--muted);
    font-family: 'DM Mono', monospace;
}

/* NAV */
.nav {
    display: flex;
    gap: 2px;
    overflow-x: auto;
    scrollbar-width: none;
}
.nav::-webkit-scrollbar { display: none; }
.nav-btn {
    background: none;
    border: none;
    color: var(--muted);
    font-family: 'DM Sans', sans-serif;
    font-size: 13px;
    font-weight: 500;
    padding: 0 14px;
    height: 56px;
    cursor: pointer;
    white-space: nowrap;
    border-bottom: 2px solid transparent;
    transition: all 0.15s;
    display: flex;
    align-items: center;
    gap: 6px;
}
.nav-btn:hover { color: var(--text); }
.nav-btn.active { color: var(--teal); border-bottom-color: var(--teal); }

/* MAIN */
.main {
    flex: 1;
    padding: 1.5rem;
    max-width: 1280px;
    margin: 0 auto;
    width: 100%;
}

/* CARDS */
.card {

```

```

    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: 12px;
    padding: 1.25rem;
    margin-bottom: 1rem;
}

.card-title {
    font-size: 12px;
    font-weight: 600;
    color: var(--muted);
    text-transform: uppercase;
    letter-spacing: 0.08em;
    margin-bottom: 1rem;
}

/* KPI ROW */
.kpi-row {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
    gap: 12px;
    margin-bottom: 1.25rem;
}

.kpi {
    background: var(--surface2);
    border: 1px solid var(--border);
    border-radius: 10px;
    padding: 1rem;
}

.kpi-label {
    font-size: 11px;
    color: var(--muted);
    font-weight: 500;
    text-transform: uppercase;
    letter-spacing: 0.06em;
    margin-bottom: 6px;
}

.kpi-value {
    font-size: 26px;
    font-weight: 700;
    font-family: 'DM Mono', monospace;
    color: var(--teal);
    line-height: 1;
}

.kpi-sub { font-size: 11px; color: var(--muted); margin-top: 4px; }

/* TABLE */
.table-wrap { overflow-x: auto; border-radius: 8px; }

```

```

table { width: 100%; border-collapse: collapse; font-size: 13px; }
thead th {
  background: var(--surface2);
  color: var(--muted);
  font-size: 11px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  padding: 10px 12px;
  text-align: left;
  white-space: nowrap;
  border-bottom: 1px solid var(--border);
}
tbody tr {
  border-bottom: 1px solid var(--border);
  transition: background 0.1s;
}
tbody tr:last-child { border-bottom: none; }
tbody tr:hover { background: var(--surface2); }
tbody td { padding: 10px 12px; color: var(--text); vertical-align: middle; }
.td-muted { color: var(--muted); font-size: 12px; }

/* BADGES */
.badge {
  display: inline-flex;
  align-items: center;
  padding: 2px 8px;
  border-radius: 20px;
  font-size: 11px;
  font-weight: 600;
  font-family: 'DM Mono', monospace;
}
.badge-tier { background: rgba(0,191,165,0.12); color: var(--teal); }
.badge-canal { background: var(--surface2); color: var(--muted); }
.badge-alto { background: rgba(16,185,129,0.12); color: var(--success); }
.badge-medio { background: rgba(245,158,11,0.12); color: var(--warn); }
.badge-baixo { background: rgba(239,68,68,0.12); color: var(--danger); }
.badge-sem { background: var(--surface2); color: var(--muted); }

/* PROGRESS BAR */
.progress-wrap { background: var(--border); border-radius: 4px; height: 6px; ov
.progress-bar { height: 100%; border-radius: 4px; transition: width 0.3s; }

/* FORM */
.form-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200p
.form-full { grid-column: 1 / -1; }
.form-group { display: flex; flex-direction: column; gap: 5px; }

```

```

.form-label { font-size: 12px; font-weight: 500; color: var(--muted); }
.form-input, .form-select, .form-textarea {
  background: var(--surface2);
  border: 1px solid var(--border);
  border-radius: 8px;
  color: var(--text);
  font-family: 'DM Sans', sans-serif;
  font-size: 13px;
  padding: 9px 12px;
  outline: none;
  transition: border-color 0.15s;
  width: 100%;
}
.form-input:focus, .form-select:focus, .form-textarea:focus {
  border-color: var(--teal);
}
.form-select option { background: var(--surface); }
.form-textarea { resize: vertical; min-height: 80px; }
.form-checkbox {
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 13px;
  cursor: pointer;
  color: var(--text);
  margin-top: 8px;
}
.form-checkbox input { accent-color: var(--teal); width: 15px; height: 15px; }

/* BUTTONS */
.btn {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 9px 18px;
  border-radius: 8px;
  font-family: 'DM Sans', sans-serif;
  font-size: 13px;
  font-weight: 600;
  cursor: pointer;
  border: none;
  transition: all 0.15s;
}
.btn-primary {
  background: var(--teal);
  color: #0a0e1a;
}

```

```

.btn-primary:hover { background: #00d4b8; }
.btn-ghost {
  background: var(--surface2);
  color: var(--text);
  border: 1px solid var(--border);
}
.btn-ghost:hover { border-color: var(--teal); color: var(--teal); }
.btn-danger { background: rgba(239,68,68,0.15); color: var(--danger); border: 1
.btn-sm { padding: 5px 12px; font-size: 12px; }
.btn-row { display: flex; gap: 8px; flex-wrap: wrap; margin-top: 1rem; }

/* TP GRID */
.tp-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(260px
.tp-card {
  background: var(--surface2);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 12px;
  cursor: pointer;
  transition: all 0.15s;
}
.tp-card:hover { border-color: var(--teal); }
.tp-card.done { border-color: rgba(16,185,129,0.4); background: rgba(16,185,129
.tp-header { display: flex; align-items: center; justify-content: space-between
.tp-code { font-family: 'DM Mono', monospace; font-size: 11px; font-weight: 600
.tp-check { width: 18px; height: 18px; border-radius: 50%; border: 2px solid va
.tp-check.done { background: var(--success); border-color: var(--success); colo
.tp-name { font-size: 13px; font-weight: 500; margin-bottom: 4px; }
.tp-bloco { font-size: 11px; font-weight: 600; }

/* MEDICO PROFILE */
.medico-header {
  display: flex;
  align-items: flex-start;
  gap: 14px;
  margin-bottom: 1rem;
}
.medico-avatar {
  width: 44px;
  height: 44px;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--teal-dim), var(--teal));
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 16px;
  font-weight: 700;

```

```

    color: #0a0e1a;
    flex-shrink: 0;
}

/* LOADING / EMPTY */
.loading {
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 3rem;
    color: var(--muted);
    font-size: 13px;
    gap: 8px;
}
.spinner {
    width: 18px; height: 18px;
    border: 2px solid var(--border);
    border-top-color: var(--teal);
    border-radius: 50%;
    animation: spin 0.7s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }

.empty {
    text-align: center;
    padding: 3rem;
    color: var(--muted);
    font-size: 13px;
}

/* SEARCH */
.search-row {
    display: flex;
    gap: 8px;
    margin-bottom: 1rem;
    flex-wrap: wrap;
}
.search-input {
    background: var(--surface2);
    border: 1px solid var(--border);
    border-radius: 8px;
    color: var(--text);
    font-family: 'DM Sans', sans-serif;
    font-size: 13px;
    padding: 8px 12px;
    outline: none;
    flex: 1;
}

```



```

    min-width: 160px;
}
.search-input:focus { border-color: var(--teal); }
.search-select {
    background: var(--surface2);
    border: 1px solid var(--border);
    border-radius: 8px;
    color: var(--text);
    font-family: 'DM Sans', sans-serif;
    font-size: 13px;
    padding: 8px 12px;
    outline: none;
    cursor: pointer;
}
.search-select:focus { border-color: var(--teal); }

/* TOAST */
.toast {
    position: fixed;
    bottom: 1.5rem;
    right: 1.5rem;
    background: var(--surface);
    border: 1px solid var(--teal);
    border-radius: 10px;
    padding: 12px 18px;
    font-size: 13px;
    font-weight: 500;
    color: var(--teal);
    z-index: 9999;
    animation: fadeUp 0.3s ease;
    box-shadow: 0 8px 32px rgba(0,0,0,0.4);
}
@keyframes fadeUp { from { opacity: 0; transform: translateY(12px); } to { opac

/* CONFIG BANNER */
.config-banner {
    background: rgba(245,158,11,0.1);
    border: 1px solid rgba(245,158,11,0.3);
    border-radius: 10px;
    padding: 1rem 1.25rem;
    margin-bottom: 1.25rem;
    font-size: 13px;
    color: var(--warn);
}
.config-banner code {
    background: rgba(245,158,11,0.15);
    padding: 1px 5px;

```

```

    border-radius: 4px;
    font-family: 'DM Mono', monospace;
    font-size: 12px;
}

/* RESPONSIVE */
@media (max-width: 640px) {
    .main { padding: 1rem; }
    .kpi-row { grid-template-columns: repeat(2, 1fr); }
    .header-title { font-size: 13px; }
}

/* TIER BAR */
.tier-row {
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 8px 0;
    border-bottom: 1px solid var(--border);
}
.tier-row:last-child { border-bottom: none; }
.tier-name { font-size: 13px; font-weight: 500; min-width: 130px; }
.tier-count { font-family: 'DM Mono', monospace; font-size: 12px; color: var(--

/* MODAL */
.modal-overlay {
    position: fixed; inset: 0;
    background: rgba(0,0,0,0.7);
    z-index: 200;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 1rem;
}
.modal {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: 14px;
    padding: 1.5rem;
    width: 100%;
    max-width: 560px;
    max-height: 90vh;
    overflow-y: auto;
}
.modal-title {
    font-size: 16px;
    font-weight: 700;

```

```

    margin-bottom: 1.25rem;
    display: flex;
    align-items: center;
    justify-content: space-between;
  }
.modal-close {
  background: none;
  border: none;
  color: var(--muted);
  font-size: 20px;
  cursor: pointer;
  line-height: 1;
}
.modal-close:hover { color: var(--text); }

/* SECTION HEADER */
.section-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 1rem;
  flex-wrap: wrap;
  gap: 8px;
}
.section-title { font-size: 17px; font-weight: 700; letter-spacing: -0.02em; }
.section-sub { font-size: 12px; color: var(--muted); margin-top: 2px; }
`;

// ——— UTILS ———
const initials = (name) => name ? name.split(" ").slice(0,2).map(w => w[0]).join(
const isConfigured = SUPABASE_URL !== "https://vumtiszgrcjzgekkpgmp.supabase.co";

function probBadge(prob) {
  const map = { Alto: "badge-alto", Médio: "badge-medio", Baixo: "badge-baixo", "
  return <span className={`badge ${map[prob]} || "badge-sem"}`>{prob} || "-"</spa
}

function Toast({ msg, onHide }) {
  useEffect(() => { const t = setTimeout(onHide, 2800); return () => clearTimeout
  return <div className="toast">✓ {msg}</div>;
}

// ——— HOOKS ———
function useSupabase(table, deps = []) {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  const fetch = useCallback(async () => {

```

```

    if (!isConfigured) { setLoading(false); return; }
    setLoading(true);
    const { data: rows } = await supabase.from(table).select("*");
    setData(rows || []);
    setLoading(false);
  }, deps);
  useEffect(() => { fetch(); }, [fetch]);
  return { data, loading, refresh: fetch };
}

```

```

// ——— DASHBOARD —————
function Dashboard() {
  const { data: visitas, loading } = useSupabase("visitas");
  const { data: medicos } = useSupabase("medicos");
  const { data: medicoTps } = useSupabase("medico_tps");

  if (!isConfigured) return <ConfigBanner />;

  const totalVisitas = visitas.length;
  const presenciais = visitas.filter(v => v.canal === "Presencial").length;
  const altoProb = visitas.filter(v => v.probabilidade === "Alto").length;
  const suporteMkt = visitas.filter(v => v.suporte_marketing).length;
  const totalTps = medicoTps.length;
  const tpsConcluidos = medicoTps.filter(t => t.concluido).length;

  const byCanal = CANAIS.reduce((acc, c) => {
    acc[c] = visitas.filter(v => v.canal === c).length;
    return acc;
  }, {});

  const byResponsavel = {};
  visitas.forEach(v => {
    if (v.responsavel && !v.responsavel.startsWith("—")) {
      byResponsavel[v.responsavel] = (byResponsavel[v.responsavel] || 0) + 1;
    }
  });

  const byTier = [1,2,3,4,5].map(t => ({
    tier: t,
    total: medicos.filter(m => m.tier === t).length,
    visitados: visitas.filter(v => {
      const m = medicos.find(md => md.nome === v.medico_nome);
      return m?.tier === t;
    }).length
  }));

  return (

```

```

<div>
  <div className="section-header">
    <div>
      <div className="section-title">Dashboard</div>
      <div className="section-sub">Visão em tempo real · atualiza com cada re
    </div>
  </div>

  {loading ? <div className="loading"><div className="spinner" /> Carregando.
  <>
    <div className="kpi-row">
      <div className="kpi">
        <div className="kpi-label">Total Visitas</div>
        <div className="kpi-value">{totalVisitas}</div>
        <div className="kpi-sub">{presenciais} presenciais</div>
      </div>
      <div className="kpi">
        <div className="kpi-label">Alta Prob.</div>
        <div className="kpi-value" style={{color:"#10b981"}}>{altoProb}</di
        <div className="kpi-sub">médicos quentes</div>
      </div>
      <div className="kpi">
        <div className="kpi-label">Suporte MKT</div>
        <div className="kpi-value" style={{color:"#f59e0b"}}>{suporteMkt}</
        <div className="kpi-sub">solicitações ativas</div>
      </div>
      <div className="kpi">
        <div className="kpi-label">TPs Concluídos</div>
        <div className="kpi-value" style={{color:"#8b5cf6"}}>{tpsConcluidos
        <div className="kpi-sub">de {totalTps} registrados</div>
      </div>
      <div className="kpi">
        <div className="kpi-label">Médicos Base</div>
        <div className="kpi-value">{medicos.length}</div>
        <div className="kpi-sub">na carteira</div>
      </div>
    </div>

    <div style={{display:"grid", gridTemplateColumns:"1fr 1fr", gap:"12px"}}
    <div className="card">
      <div className="card-title">Visitas por Canal</div>
      {Object.entries(byCanal).map(([canal, count]) => (
        <div key={canal} style={{marginBottom:10}}>
          <div style={{display:"flex",justifyContent:"space-between",font
            <span>{canal}</span>
            <span style={{fontFamily:"DM Mono",color:"var(--teal)"}>{cou
          </div>

```



```
// — VISITAS —————
function Visitas() {
  const { data: visitas, loading, refresh } = useSupabase("visitas");
  const { data: medicos } = useSupabase("medicos");
  const [showForm, setShowForm] = useState(false);
  const [search, setSearch] = useState("");
  const [filterResp, setFilterResp] = useState("");
  const [toast, setToast] = useState(null);

  const sorted = [...visitas]
    .sort((a,b) => new Date(b.data_contato) - new Date(a.data_contato))
    .filter(v => {
      const q = search.toLowerCase();
      return (!q || v.medico_nome?.toLowerCase().includes(q) || v.obs?.toLowerCase()
        && (!filterResp || v.responsavel === filterResp));
    });

  const handleSave = async (form) => {
    if (!isConfigured) return;
    const medico = medicos.find(m => m.nome === form.medico_nome);
    await supabase.from("visitas").insert([
      {
        ...form,
        medico_id: medico?.id || null,
        referidor: form.referidor === "Sim",
        suporte_marketing: form.suporte_marketing === "Sim",
      }
    ]);
    setToast("Visita registrada!");
    setShowForm(false);
    refresh();
  };

  return (
    <div>
      <div className="section-header">
        <div>
          <div className="section-title">Registro de Visitas</div>
          <div className="section-sub">{visitas.length} visitas registradas</div>
        </div>
        <button className="btn btn-primary" onClick={() => setShowForm(true)}>+ N
      </div>

      {!isConfigured && <ConfigBanner />}

      <div className="search-row">
        <input className="search-input" placeholder="Buscar médico ou observação."
        <select className="search-select" value={filterResp} onChange={e => setFi
```

```

    <option value="">Todos responsáveis</option>
    {RESPONSABLEIS.filter(r => !r.startsWith("—")).map(r => <option key={r}
  </select>
</div>

{loading ? <div className="loading"><div className="spinner" /> Carregando.
  <div className="card" style={{padding:0}}>
    <div className="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Data</th>
            <th>Responsável</th>
            <th>Médico</th>
            <th>Canal</th>
            <th>Último TP</th>
            <th>Prob.</th>
            <th>MKT</th>
            <th>Observação</th>
          </tr>
        </thead>
        <tbody>
          {sorted.length === 0 ? (
            <tr><td colSpan={8}><div className="empty">Nenhuma visita regis
          ) : sorted.map(v => (
            <tr key={v.id}>
              <td className="td-muted" style={{fontFamily:"DM Mono",fontSiz
                {v.data_contato ? new Date(v.data_contato + 'T00:00:00').to
              </td>
              <td>{v.responsavel || "—"}</td>
              <td style={{fontWeight:500}}>{v.medico_nome}</td>
              <td><span className="badge badge-canal">{v.canal || "—"}</spa
              <td className="td-muted">{v.ultimo_tp || "—"}</td>
              <td>{v.probabilidade ? probBadge(v.probabilidade) : "—"}</td>
              <td style={{color: v.suporte_marketing ? "var(--teal)" : "var
                {v.suporte_marketing ? "✓ Sim" : "—"}
              </td>
              <td className="td-muted" style={{maxWidth:220, overflow:"hidd
                {v.obs || "—"}
              </td>
            </tr>
          )}}
        </tbody>
      </table>
    </div>
  </div>
)}

```



```

    {showForm && (
      <VisitaModal
        medicos={medicos}
        onSave={handleSave}
        onClose={() => setShowForm(false)}
      />
    )}
    {toast && <Toast msg={toast} onHide={() => setToast(null)} />}
  </div>
);
}

```

```

function VisitaModal({ medicos, onSave, onClose }) {
  const [form, setForm] = useState({
    data_contato: new Date().toISOString().slice(0,10),
    responsavel: "",
    medico_nome: "",
    referidor: "Não",
    especialidade: "",
    canal: "",
    ultimo_tp: "",
    probabilidade: "",
    valor_esperado: "",
    suporte_marketing: "Não",
    acao_mkt: "",
    acao_prevista: "",
    obs: "",
    mes: "JUNHO 26",
    fios_esg_estoque: "",
    proc_marcados_esg: "",
    baloes_estoque: "",
    proc_marcados_balao: "",
  });
  const set = (k, v) => setForm(f => ({...f, [k]: v}));

  return (
    <div className="modal-overlay" onClick={e => e.target === e.currentTarget &&
      <div className="modal">
        <div className="modal-title">
          Nova Visita
          <button className="modal-close" onClick={onClose}>×</button>
        </div>
        <div className="form-grid">
          <div className="form-group">
            <label className="form-label">Data do Contato *</label>
            <input className="form-input" type="date" value={form.data_contato} o

```

```

</div>
<div className="form-group">
  <label className="form-label">Responsável *</label>
  <select className="form-select" value={form.responsavel} onChange={e}
    <option value="">Selecione...</option>
    {RESPONSABLEIS.filter(r => !r.startsWith("—")).map(r => <option key
  </select>
</div>
<div className="form-group">
  <label className="form-label">Médico *</label>
  <input
    className="form-input"
    list="medicos-list"
    value={form.medico_nome}
    onChange={e => set("medico_nome", e.target.value)}
    placeholder="Nome do médico"
  />
  <datalist id="medicos-list">
    {medicos.map(m => <option key={m.id} value={m.nome} />)}
  </datalist>
</div>
<div className="form-group">
  <label className="form-label">Canal</label>
  <select className="form-select" value={form.canal} onChange={e => set
    <option value="">Selecione...</option>
    {CANALS.map(c => <option key={c}>{c}</option>)}
  </select>
</div>
<div className="form-group">
  <label className="form-label">Referidor?</label>
  <select className="form-select" value={form.referidor} onChange={e =>
    <option>Não</option>
    <option>Sim</option>
  </select>
</div>
<div className="form-group">
  <label className="form-label">Especialidade</label>
  <input className="form-input" value={form.especialidade} onChange={e
</div>
<div className="form-group">
  <label className="form-label">Último TP</label>
  <select className="form-select" value={form.ultimo_tp} onChange={e =>
    <option value="">—</option>
    {Array.from({length:23}, (_,i)=>`TP${i+1}`).map(tp => <option key={t
  </select>
</div>
<div className="form-group">

```

```

    <label className="form-label">Probabilidade de Fechar</label>
    <select className="form-select" value={form.probabilidade} onChange={
      <option value="">--</option>
      {PROBABILIDADES.map(p => <option key={p}><p></option>)}
    }
  </select>
</div>
<div className="form-group">
  <label className="form-label">$ Esperado no Mês</label>
  <input className="form-input" type="number" value={form.valor_esperad
</div>
<div className="form-group">
  <label className="form-label">Fios ESG em Estoque</label>
  <input className="form-input" type="number" value={form.fios_esg_esto
</div>
<div className="form-group">
  <label className="form-label">Proc. Marcados (ESG)</label>
  <input className="form-input" type="number" value={form.proc_marcados
</div>
<div className="form-group">
  <label className="form-label">Balões em Estoque</label>
  <input className="form-input" type="number" value={form.baloes_estoqu
</div>
<div className="form-group form-full">
  <label className="form-label">Ação Prevista</label>
  <input className="form-input" value={form.acao_prevista} onChange={e
</div>
<div className="form-group form-full">
  <label className="form-label">Suporte Marketing?</label>
  <select className="form-select" value={form.suporte_marketing} onChan
    <option>Não</option>
    <option>Sim</option>
  </select>
</div>
{form.suporte_marketing === "Sim" && (
  <div className="form-group form-full">
    <label className="form-label">Ação MKT</label>
    <input className="form-input" value={form.acao_mkt} onChange={e =>
  </div>
)}
<div className="form-group form-full">
  <label className="form-label">Observações</label>
  <textarea className="form-textarea" value={form.obs} onChange={e => s
</div>
</div>
<div className="btn-row">
  <button className="btn btn-primary" onClick={() => {
    if (!form.data_contato || !form.medico_nome) return alert("Preencha d

```

```

        onSave(form);
    }}>Salvar Visita</button>
    <button className="btn btn-ghost" onClick={onClose}>Cancelar</button>
  </div>
</div>
</div>
);
}

// — MÉDICOS —————
function Medicos() {
  const { data: medicos, loading } = useSupabase("medicos");
  const { data: tps } = useSupabase("medico_tps");
  const [search, setSearch] = useState("");
  const [filterTier, setFilterTier] = useState("");
  const [selected, setSelected] = useState(null);

  const filtered = medicos.filter(m => {
    const q = search.toLowerCase();
    return (!q || m.nome?.toLowerCase().includes(q) || m.estado?.toLowerCase().in
      && (!filterTier || String(m.tier) === filterTier);
  }).sort((a,b) => (a.tier||99) - (b.tier||99));

  const getMedicoProgress = (medicoId) => {
    const mTps = tps.filter(t => t.medico_id === medicoId);
    const done = mTps.filter(t => t.concluido).length;
    return { done, total: mTps.length, pct: mTps.length ? Math.round(done/mTps.le
  };

  return (
    <div>
      <div className="section-header">
        <div>
          <div className="section-title">Carteira de Médicos</div>
          <div className="section-sub">{medicos.length} médicos cadastrados</div>
        </div>
      </div>

      {!isConfigured && <ConfigBanner />}

      <div className="search-row">
        <input className="search-input" placeholder="Buscar médico, estado..." va
        <select className="search-select" value={filterTier} onChange={e => setFi
          <option value="">Todos os tiers</option>
          {[ "ST", "1", "2", "3", "4", "5" ].map(t => <option key={t} value={t}>Tier {t}
        </select>
      </div>
    </div>
  );
}

```

```

{loading ? <div className="loading"><div className="spinner" /> Carregando.
  <div className="card" style={{padding:0}}>
    <div className="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Médico</th>
            <th>Tier</th>
            <th>Estado</th>
            <th>Clínica</th>
            <th>Account</th>
            <th>Ativação TPs</th>
          </tr>
        </thead>
        <tbody>
          {filtered.map(m => {
            const prog = getMedicoProgress(m.id);
            return (
              <tr key={m.id} style={{cursor:"pointer"}} onClick={() => setS
                <td>
                  <div style={{display:"flex",alignItems:"center",gap:10}}>
                    <div className="medico-avatar" style={{width:32,height:
                      {initials(m.nome)}
                    </div>
                    <span style={{fontWeight:500}}>{m.nome}</span>
                  </div>
                </td>
                <td>
                  <span className="badge badge-tier" style={{background:`${
                    {m.tier === "ST" ? "STAR" : m.tier ? `T${m.tier}` : "-"
                  </span>
                </td>
                <td className="td-muted">{m.estado || "-"}</td>
                <td className="td-muted" style={{maxWidth:160,overflow:"hid
                <td className="td-muted">{m.account || "-"}</td>
                <td style={{minWidth:120}}>
                  {prog.total > 0 ? (
                    <div>
                      <div style={{fontSize:11,color:"var(--muted)",marginB
                      <div className="progress-wrap">
                        <div className="progress-bar" style={{
                          width:`${prog.pct}%`,
                          background: prog.pct >= 70 ? "var(--success)" : p
                        </div>
                      </div>
                    </div>
                  </td>
            </div>
          }

```

```

        ) : <span className="td-muted">—</span>
      </td>
    </tr>
  </tbody>
</table>
</div>
</div>
))
{selected && <MedicoModal medico={selected} onClose={() => setSelected(null)}
</div>
);
}

```

```

function MedicoModal({ medico, onClose }) {
  return (
    <div className="modal-overlay" onClick={e => e.target === e.currentTarget &&
      <div className="modal">
        <div className="modal-title">
          <div style={{display:"flex",alignItems:"center",gap:12}}>
            <div className="medico-avatar">{initials(medico.nome)}</div>
            <div>
              <div>{medico.nome}</div>
              <div style={{fontSize:12,fontWeight:400,color:"var(--muted)"}>
                {medico.clinica || medico.estado || "—"}
              </div>
            </div>
          </div>
          <div>
            <button className="modal-close" onClick={onClose}>×</button>
          </div>
        </div>
        <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:10,fontSize:12}}>
          {
            [
              ["Tier", medico.tier ? `T${medico.tier}` : "—"],
              ["Estado", medico.estado || "—"],
              ["Região", medico.regiao || "—"],
              ["Account", medico.account || "—"],
              ["Perfil", medico.perfil_medico || "—"],
              ["Estágio BSCI", medico.estagio_bsci || "—"],
            ].map(([label, val]) => (
              <div key={label} style={{background:"var(--surface2)",borderRadius:8,padding:10}}>
                <div style={{fontSize:11,color:"var(--muted)",marginBottom:4}}>{label}</div>
                <div style={{fontWeight:500}}>{val}</div>
              </div>
            ))}
        </div>
        {medico.gaps && (

```



```

        medico_id: medicoId, tp_id: tpId, concluido: true,
        data_conclusao: new Date().toISOString().slice(0,10)
    }]);
}
setToast("TP atualizado!");
refresh();
};

return (
    <div>
        <div className="section-header">
            <div>
                <div className="section-title">Ativação de TPs</div>
                <div className="section-sub">Acompanhe os 23 touchpoints por médico</div>
            </div>
            <select className="search-select" value={filterTier} onChange={e => setFi
                <option value="">Todos os tiers</option>
                {[ "ST", "1", "2", "3", "4", "5"].map(t => <option key={t} value={t}>Tier {t}
            </select>
        </div>

        {!isConfigured && <ConfigBanner />}
        {loading ? <div className="loading"><div className="spinner" /> Carregando.
            <div>
                {selectedMedico ? (
                    <div>
                        <div style={{display:"flex",alignItems:"center",gap:12,marginBottom
                            <button className="btn btn-ghost btn-sm" onClick={() => setSelect
                            <div className="medico-avator" style={{width:36,height:36,fontSiz
                                <div>
                                    <div style={{fontWeight:600,fontSize:15}}>{selectedMedico.nome}
                                    <div style={{fontSize:12,color:"var(--muted)"}>
                                        {selectedMedico.tier ? `T${selectedMedico.tier}` : ""} · {sel
                                    </div>
                                </div>
                            </div>
                        </div>

                    <div className="tp-grid">
                        {tpsDef.map(tp => {
                            const status = getTpStatus(selectedMedico.id, tp.id);
                            const done = status?.concluido;
                            return (
                                <div
                                    key={tp.id}
                                    className={`tp-card ${done ? "done" : ""}`}
                                    onClick={() => toggleTp(selectedMedico.id, tp.id)}
                                >

```



```

<div className="tp-header">
  <span className="tp-code" style={{color: BLOCOS_COLOR[tp.
    {tp.codigo}
  </span>
  <div className={`tp-check ${done ? "done" : ""}`}>
    {done ? "✓" : ""}
  </div>
</div>
<div className="tp-name">{tp.nome}</div>
<div className="tp-bloco" style={{color: BLOCOS_COLOR[tp.bl
  {tp.bloco}
</div>
{tp.o_que_validar && (
  <div style={{fontSize:11,color:"var(--muted)",marginTop:6
    {tp.o_que_validar}
  </div>
  )}
</div>
  );
  )}}
</div>
</div>
) : (
  <div className="card" style={{padding:0}}>
    <div className="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Médico</th>
            <th>Tier</th>
            <th>Estado</th>
            <th>TPs Concluídos</th>
            <th>Alerta</th>
          </tr>
        </thead>
        <tbody>
          {medicosFilt.map(m => {
            const prog = getProgress(m.id);
            const alerta = prog.pct >= 70 ? { icon: "🟢", label: "Alta
              prog.pct >= 40 ? { icon: "🟡", label: "Em progresso" } :
              { icon: "🔴", label: "Baixa ativação" };
            return (
              <tr key={m.id} style={{cursor:"pointer"}} onClick={() =>
                <td style={{fontWeight:500}}>{m.nome}</td>
                <td>
                  <span className="badge badge-tier" style={{background
                    {m.tier === "ST" ? "STAR" : m.tier ? `T${m.tier}` :

```

```

        </span>
      </td>
      <td className="td-muted">{m.estado || "-"}</td>
      <td style={{minWidth:160}}>
        <div style={{fontSize:11,color:"var(--muted)",marginB
          {prog.done}/{prog.total > 0 ? prog.total : 23} · {p
        </div>
        <div className="progress-wrap">
          <div className="progress-bar" style={{
            width:`${prog.pct}%`,
            background: prog.pct >= 70 ? "var(--success)" : p
          }} />
        </div>
      </td>
      <td style={{fontSize:12}}>{alerta.icon} {alerta.label}<
    </tr>
  );
  }}}
</tbody>
</table>
</div>
</div>
  )}
</div>
  )}
  {toast && <Toast msg={toast} onHide={() => setToast(null)} />}
</div>
);
}

```

```

// — INTELIGÊNCIA —————
function Inteligencia() {
  const { data: visitas, loading } = useSupabase("visitas");
  const { data: medicos } = useSupabase("medicos");

  const semResp = (resp) => {
    if (!resp) return true;
    return ["Cibele","Catharina","Sarah","Guilherme","Júlia"].some(n => resp.toLo
  };

  const medicosSemTP = visitas.filter(v =>
    v.medico_nome && !v.ultimo_tp && !semResp(v.responsavel)
  ).map(v => v.medico_nome);
  const uniqueSemTP = [...new Set(medicosSemTP)];

  const tierCoverage = [1,2,3,4,5].map(tier => {
    const mTier = medicos.filter(m => m.tier === tier);

```

```

const visitados = mTier.filter(m =>
  visitas.some(v => v.medico_nome === m.nome && v.canal === "Presencial")
);
const pct = mTier.length ? Math.round(visitados.length / mTier.length * 100)
const alert = pct === 0 ? "🔴 SEM PRESENCIAL" : pct >= 60 ? "🟢 BOA COBERTURA"
return { tier, total: mTier.length, visitados: visitados.length, pct, alert }
});

const medicosSemPresencial = medicos.filter(m =>
  !visitas.some(v => v.medico_nome === m.nome && v.canal === "Presencial")
);

const highProb = visitas.filter(v => v.probabilidade === "Alto");

return (
  <div>
    <div className="section-header">
      <div>
        <div className="section-title">Inteligência Estratégica</div>
        <div className="section-sub">Análise cruzada · atualiza automaticamente</div>
      </div>
    </div>

    {!isConfigured && <ConfigBanner />}
    {loading ? <div className="loading"><div className="spinner" /> Carregando.
    <>
      <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:12;margin
      <div className="card">
        <div className="card-title">⚡ Sem TP Registrado</div>
        {uniqueSemTP.length === 0 ? (
          <div className="empty">Todos os médicos têm TP registrado 🎉</div>
        ) : uniqueSemTP.map(n => (
          <div key={n} style={{fontSize:13,padding:"6px 0",borderBottom:"1px
            <span style={{color:"var(--danger)"}}>●</span> {n}
          </div>
        ))}
      </div>
      <div className="card">
        <div className="card-title">🔥 Alta Probabilidade</div>
        {highProb.length === 0 ? (
          <div className="empty">Nenhum médico com alta probabilidade.</div>
        ) : highProb.map(v => (
          <div key={v.id} style={{fontSize:13,padding:"6px 0",borderBottom:
            <div style={{fontWeight:500}}>{v.medico_nome}</div>
            <div style={{fontSize:11,color:"var(--muted)"}}>{v.responsavel}
          </div>
        ))}
    </div>
  </div>

```

```

    </div>
  </div>

  <div className="card">
    <div className="card-title">► Cobertura por Tier</div>
    <div className="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Tier</th>
            <th>Total Carteira</th>
            <th>Na Planilha</th>
            <th>% Cobertura</th>
            <th>Alerta</th>
            <th>Médicos Faltantes</th>
          </tr>
        </thead>
        <tbody>
          {tierCoverage.map(t => (
            <tr key={t.tier}>
              <td><span className="badge badge-tier" style={{background:`
              <td style={{fontFamily:"DM Mono"}}>{t.total}</td>
              <td style={{fontFamily:"DM Mono"}}>{t.visitados}</td>
              <td>
                <div style={{display:"flex",alignItems:"center",gap:8}}>
                  <div className="progress-wrap" style={{flex:1}}>
                    <div className="progress-bar" style={{
                      width:`${t.pct}%`,
                      background: t.pct >= 60 ? "var(--success)" : t.pct
                    }} />
                  </div>
                  <span style={{fontFamily:"DM Mono",fontSize:12,minWidth
                </div>
              </td>
              <td style={{fontSize:12}}>{t.alert}</td>
              <td style={{fontFamily:"DM Mono",fontSize:12,color:"var(--d
            </tr>
          )}}
        </tbody>
      </table>
    </div>
  </div>

  <div className="card">
    <div className="card-title">📅 Médicos sem Visita Presencial</div>
    <div style={{display:"flex",flexWrap:"wrap",gap:6}}>
      {medicosSemPresencial.map(m => (

```

```

        <span key={m.id} style={{
          background:"var(--surface2)",border:"1px solid var(--border)",
          borderRadius:6,padding:"4px 10px",fontSize:12,display:"flex",ga
        }}>
          <span style={{color:TIER_COLOR[m.tier]||"#888",fontFamily:"DM M
            {m.nome}
          </span>
        </span>
      )}}
      {medicosSemPresencial.length === 0 && (
        <span style={{fontSize:13,color:"var(--muted)"}>Todos os médicos
      </span>
    </div>
  </div>
</>
)}
</div>
);
}

// ——— CONFIG BANNER —————
function ConfigBanner() {
  return (
    <div className="config-banner">
      <strong>⚠ Configuração necessária</strong><br/>
      Substitua <code>https://vumtiszgrcjzgekkpgmp.supabase.co</code> e <code>sb_
      Depois rode o arquivo <strong>supabase_schema.sql</strong> no SQL Editor do
    </div>
  );
}

// ——— CARTILHA —————
function Cartilha() {
  const { data: tps, loading } = useSupabase("tps_definicoes");
  const [filterBloco, setFilterBloco] = useState("");
  const blocos = [...new Set(tps.map(t => t.bloco))];
  const filtered = tps.filter(t => !filterBloco || t.bloco === filterBloco);

  return (
    <div>
      <div className="section-header">
        <div>
          <div className="section-title">Cartilha de TPs</div>
          <div className="section-sub">Consulta rápida antes da visita</div>
        </div>
        <select className="search-select" value={filterBloco} onChange={e => setF
          <option value="">Todos os blocos</option>
          {blocos.map(b => <option key={b}>{b}</option>)}
        </select>
      </div>
    </div>
  );
}

```

```

        </select>
    </div>

    {!isConfigured && <ConfigBanner />}
    {loading ? <div className="loading"><div className="spinner" /> Carregando.
    <div className="tp-grid">
        {filtered.map(tp => (
            <div key={tp.id} className="tp-card" style={{cursor:"default"}}>
                <div className="tp-header">
                    <span className="tp-code" style={{color: BLOCOS_COLOR[tp.bloco] |
                        {tp.codigo}}
                    </span>
                    <span className="badge" style={{
                        background:`${BLOCOS_COLOR[tp.bloco] || "var(--teal)"}18`,
                        color: BLOCOS_COLOR[tp.bloco] || "var(--teal)",
                        fontSize:10
                    }}>{tp.bloco}</span>
                </div>
                <div className="tp-name">{tp.nome}</div>
                {tp.objetivo && (
                    <div style={{fontSize:11,color:"var(--muted)",marginTop:6,lineHei
                        <strong style={{color:"var(--text)"},"}}>Objetivo:</strong> {tp.ob
                    </div>
                )}
                {tp.o_que_validar && (
                    <div style={{
                        fontSize:11,color:"var(--teal)",marginTop:8,lineHeight:1.5,
                        background:"rgba(0,191,165,0.05)",borderRadius:6,padding:"6px 8
                    }}>
                        <strong>Validar:</strong> {tp.o_que_validar}
                    </div>
                )}
            </div>
        )})
    </div>
    );
}

// ——— SETUP —————
function Setup() {
    return (
        <div>
            <div className="section-header">
                <div>
                    <div className="section-title">Configuração Inicial</div>

```

```

        <div className="section-sub">Siga os passos para conectar o app ao Supa
    </div>
</div>
<div className="card">
    <div className="card-title"><img alt="Notepad icon" data-bbox="465 125 485 140"/> Passo a Passo</div>
    {[
        { n:"1", title:"Crie o projeto no Supabase", desc:"Acesse app.supabase.
        { n:"2", title:"Rode o SQL Schema", desc:'No Supabase: SQL Editor → New
        { n:"3", title:"Copie as credenciais", desc:'Em Settings → API: copie o
        { n:"4", title:"Configure o app", desc:'No topo do arquivo ESGIntellige
        { n:"5", title:"Deploy no Vercel", desc:'Crie um repositório GitHub com
    ].map(step => (
        <div key={step.n} style={{display:"flex",gap:14,padding:"14px 0",border
            <div style={{
                width:28,height:28,borderRadius:"50%",background:"var(--teal)",
                color:"#0a0e1a",display:"flex",alignItems:"center",justifyContent:"
                fontWeight:700,fontSize:13,flexShrink:0
            }}>{step.n}</div>
        <div>
            <div style={{fontWeight:600,marginBottom:4}}>{step.title}</div>
            <div style={{fontSize:13,color:"var(--muted)",lineHeight:1.6}}>{ste
        </div>
    </div>
    )})
</div>

<div className="card">
    <div className="card-title"><img alt="Database icon" data-bbox="465 565 485 580"/> Estrutura do Banco de Dados</div>
    {[
        { table:"medicos", desc:"55+ médicos com tier, estado, clínica, account
        { table:"visitas", desc:"Log de todas as visitas da equipe (equivale à
        { table:"tps_definicoes", desc:"23 TPs da Cartilha com objetivo e o que
        { table:"medico_tps", desc:"Status de cada TP por médico (concluído / p
    ].map(t => (
        <div key={t.table} style={{display:"flex",justifyContent:"space-between
            <code style={{fontFamily:"DM Mono",color:"var(--teal)",fontSize:12}}>
            <span style={{color:"var(--muted)"}}>{t.desc}</span>
        </div>
    )})
</div>
</div>
);
}

```

```
// — METAS MENSAIS —
```

```
const MESES = [
```

```

    "JANEIRO 26", "FEVEREIRO 26", "MARÇO 26", "ABRIL 26", "MAIO 26", "JUNHO 26",
    "JULHO 26", "AGOSTO 26", "SETEMBRO 26", "OUTUBRO 26", "NOVEMBRO 26", "DEZEMBRO 26"
  ];
const TEAM = ["Marcell", "Fernanda", "Ana Marieta", "Inside Sales", "Cibele", "Guilher

function Metas() {
  const { data: visitas } = useSupabase("visitas");
  const { data: medicoTps } = useSupabase("medico_tps");
  const [metas, setMetas] = useState([]);
  const [loading, setLoading] = useState(true);
  const [mes, setMes] = useState("JUNHO 26");
  const [editing, setEditing] = useState(null);
  const [toast, setToast] = useState(null);

  const fetchMetas = useCallback(async () => {
    if (!isConfigured) { setLoading(false); return; }
    const { data } = await supabase.from("metas").select("*").eq("mes", mes);
    setMetas(data || []);
    setLoading(false);
  }, [mes]);

  useEffect(() => { fetchMetas(); }, [fetchMetas]);

  const getMeta = (resp) => metas.find(m => m.responsavel === resp) || {
    responsavel: resp, mes,
    meta_visitas: 0, meta_presenciais: 0,
    meta_procedimentos: 0, meta_receita: 0, meta_tps: 0
  };

  const getReal = (resp) => {
    const v = visitas.filter(vi => vi.responsavel === resp && vi.mes === mes);
    const tpsDone = medicoTps.filter(t => t.concluido).length;
    return {
      visitas: v.length,
      presenciais: v.filter(vi => vi.canal === "Presencial").length,
      procedimentos: v.reduce((s, vi) => s + (vi.proc_marcados_esg || 0), 0),
      receita: v.reduce((s, vi) => s + (vi.valor_esperado || 0), 0),
      tps: tpsDone,
    };
  };

  const saveMeta = async (form) => {
    if (!isConfigured) return;
    const existing = metas.find(m => m.responsavel === form.responsavel);
    if (existing) {
      await supabase.from("metas").update(form).eq("id", existing.id);
    } else {

```



```

    await supabase.from("metas").insert([form]);
  }
  setToast("Meta salva!");
  setEditing(null);
  fetchMetas();
};

const pct = (real, meta) => meta > 0 ? Math.min(Math.round((real / meta) * 100)
const color = (p) => p >= 100 ? "var(--success)" : p >= 60 ? "var(--warn)" : "v

const totais = TEAM.reduce((acc, r) => {
  const real = getReal(r);
  const meta = getMeta(r);
  acc.visitas_real += real.visitas;
  acc.presenciais_real += real.presenciais;
  acc.proc_real += real.procedimentos;
  acc.receita_real += real.receita;
  acc.visitas_meta += meta.meta_visitass;
  acc.presenciais_meta += meta.meta_presenciais;
  acc.proc_meta += meta.meta_procedimentos;
  acc.receita_meta += meta.meta_receita;
  return acc;
}, { visitas_real:0, presenciais_real:0, proc_real:0, receita_real:0,
  visitas_meta:0, presenciais_meta:0, proc_meta:0, receita_meta:0 });

return (
  <div>
    <div className="section-header">
      <div>
        <div className="section-title">Metas Mensais</div>
        <div className="section-sub">Realizado vs. meta por responsável</div>
      </div>
      <select className="search-select" value={mes} onChange={e => setMes(e.target.value)}>
        {MESES.map(m => <option key={m}>{m}</option>)}
      </select>
    </div>

    {!isConfigured && <ConfigBanner />}

    {/* KPIs do time */}
    <div className="kpi-row" style={{marginBottom:"1.25rem"}}>
      {[
        { label:"Visitas Time", real: totais.visitas_real, meta: totais.visitas_meta, pct: pct, color: color },
        { label:"Presenciais", real: totais.presenciais_real, meta: totais.presenciais_meta, pct: pct, color: color },
        { label:"Procedimentos", real: totais.proc_real, meta: totais.proc_meta, pct: pct, color: color },
        { label:"Receita Esperada $", real: totais.receita_real, meta: totais.receita_meta, pct: pct, color: color }
      ].map(k => {

```

```
const p = pct(k.real, k.meta);
return (
  <div className="kpi" key={k.label}>
    <div className="kpi-label">{k.label}</div>
    <div className="kpi-value" style={{color: color(p), fontSize:22}}>
      {k.money ? `$$${k.real.toLocaleString("pt-BR")}` : k.real}
    </div>
    <div style={{marginTop:6}}>
      <div className="progress-wrap">
        <div className="progress-bar" style={{width:`${p}%`, background
        </div>
        <div style={{fontSize:11,color:"var(--muted)",marginTop:3}}>
          {p}% de {k.money ? `$$${k.meta.toLocaleString("pt-BR")}` : k.met
        </div>
      </div>
    </div>
  );
  )}
</div>
```

```

{ /* Tabela por responsável */ }
<div className="card" style={{padding:0}}>
  <div className="table-wrap">
    <table>
      <thead>
        <tr>
          <th>Responsável</th>
          <th>Visitas</th>
          <th>Presenciais</th>
          <th>Procedimentos</th>
          <th>Receita $</th>
          <th>TPs</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {TEAM.map(resp => {
          const real = getReal(resp);
          const meta = getMeta(resp);
          return (
            <tr key={resp}>
              <td style={{fontWeight:600}}>{resp}</td>
              {[
                [real.visitas, meta.meta_visitas],
                [real.presenciais, meta.meta_presenciais],
                [real.procedimentos, meta.meta_procedimentos],
                [real.receita, meta.meta_receita, true],

```

```

    [real.tps, meta.meta_tps],
  ].map(([r, m, money], i) => {
    const p = pct(r, m);
    return (
      <td key={i} style={{minWidth:100}}>
        <div style={{fontSize:13,fontWeight:500,marginBottom:3}}
          <span style={{color: color(p)}}>
            {money ? `$$${r.toLocaleString("pt-BR")}` : r}
          </span>
          <span style={{color:"var(--muted)",fontSize:11}}> / {
        </div>
        <div className="progress-wrap">
          <div className="progress-bar" style={{width:`${p}%`,
        </div>
        </td>
      </td>
    );
  }}
</td>
</tr>
);
}}}
</tbody>
</table>
</div>
</div>

{editing && (
  <MetaModal
    meta={editing}
    mes={mes}
    onSave={saveMeta}
    onClose={() => setEditing(null)}
  />
)}
{toast && <Toast msg={toast} onHide={() => setToast(null)} />}
</div>
);
}

function MetaModal({ meta, mes, onSave, onClose }) {
  const [form, setForm] = useState({
    responsavel: meta.responsavel,
    mes,

```

```

    meta_visitas: meta.meta_visitas || 0,
    meta_presenciais: meta.meta_presenciais || 0,
    meta_procedimentos: meta.meta_procedimentos || 0,
    meta_receita: meta.meta_receita || 0,
    meta_tps: meta.meta_tps || 0,
  });
  const set = (k, v) => setForm(f => ({...f, [k]: Number(v)}));

  return (
    <div className="modal-overlay" onClick={e => e.target === e.currentTarget} &&
      <div className="modal">
        <div className="modal-title">
          Meta - {meta.responsavel}
          <button className="modal-close" onClick={onClose}>✕</button>
        </div>
        <div style={{fontSize:12,color:"var(--muted)",marginBottom:"1rem"}}>{mes}</div>
        <div className="form-grid">
          {[
            ["meta_visitas", "Meta Visitas"],
            ["meta_presenciais", "Meta Presenciais"],
            ["meta_procedimentos", "Meta Procedimentos (ESG)"],
            ["meta_receita", "Meta Receita ($)"],
            ["meta_tps", "Meta TPs Concluídos"],
          ].map(([key, label]) => (
            <div className="form-group" key={key}>
              <label className="form-label">{label}</label>
              <input
                className="form-input"
                type="number"
                value={form[key]}
                onChange={e => set(key, e.target.value)}
              />
            </div>
          ))}
        </div>
        <div className="btn-row">
          <button className="btn btn-primary" onClick={() => onSave(form)}>Salvar</button>
          <button className="btn btn-ghost" onClick={onClose}>Cancelar</button>
        </div>
      </div>
    );
  }

  // — APP ROOT —————
  const TABS = [
    { id:"dashboard", label:"Dashboard", icon:"◆" },

```

```

    { id:"visitas",    label:"Visitas",    icon:"👤" },
    { id:"medicos",    label:"Médicos",    icon:"⚕️" },
    { id:"ativacao",   label:"TPs",        icon:"📋" },
    { id:"intel",      label:"Inteligência", icon:"💡" },
    { id:"cartilha",   label:"Cartilha",   icon:"📄" },
    { id:"metas",      label:"Metas",      icon:"🎯" },
    { id:"setup",      label:"Setup",      icon:"⚙️" },
  ];

export default function App() {
  const [tab, setTab] = useState("setup");

  return (
    <div className="app">
      <style>{css}</style>
      <header className="header">
        <div className="header-brand">
          <div className="header-logo">E</div>
          <div>
            <div className="header-title">ESG Intelligence Hub</div>
            <div className="header-sub">Boston Scientific · Endura</div>
          </div>
        </div>
        <nav className="nav">
          {TABS.map(t => (
            <button
              key={t.id}
              className={`nav-btn ${tab === t.id ? "active" : ""}`}
              onClick={() => setTab(t.id)}
            >
              {t.icon} {t.label}
            </button>
          ))}
        </nav>
      </header>
      <main className="main">
        {tab === "dashboard" && <Dashboard />}
        {tab === "visitas"    && <Visitas />}
        {tab === "medicos"    && <Medicos />}
        {tab === "ativacao"   && <Ativacao />}
        {tab === "intel"      && <Inteligencia />}
        {tab === "cartilha"   && <Cartilha />}
        {tab === "metas"      && <Metas />}
        {tab === "setup"      && <Setup />}
      </main>
    </div>
  );
};

```

}

// — METAS (injected) —————